

Pattern Recognition 2024: Homework 4

周韧平– zrp21@mails.tsinghua.edu.cn

October 11, 2024

Problem 1

(i)

$$\begin{aligned}\mathcal{L} &= \frac{1}{N} \log \prod_{i=1}^N \prod_{k=1}^K P(y_i^{(k)} = 1 | \mathbf{x}_i; \mathbf{W})^{\mathbf{1}(y_i^{(k)}=1)} \\ &= \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \mathbf{1}(y_i^{(k)} = 1) \log P(y_i^{(k)} = 1 | \mathbf{x}_i; \mathbf{W}) \\ &= \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \mathbf{1}(y_i^{(k)} = 1) \log \frac{e^{\mathbf{w}_k^T \mathbf{x}_i}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x}_i}} \\ z_k &= \frac{e^{\mathbf{w}_k^T \mathbf{x}_i}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x}_i}}, \quad \frac{\partial z_k}{\partial \mathbf{w}_k} = (1 - z_k) z_k \mathbf{x}_i, \quad \frac{\partial z_k}{\partial \mathbf{w}_j} = -z_k z_j \mathbf{x}_i (j \neq k) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}_j} &= \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_i^{(k)} \frac{\partial \log z_k}{\partial \mathbf{w}_j} \\ &= \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i (y_i^{(j)} (1 - z_j) z_j \frac{1}{z_j} - \sum_{k=1, k \neq j}^K y_i^{(k)} \frac{1}{z_k} z_k z_j) \\ &= \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i (y_i^{(j)} - z_j \sum_{k=1}^K y_i^{(k)}) \\ &= \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i (y_i^{(j)} - \frac{e^{\mathbf{w}_j^T \mathbf{x}_i}}{\sum_{k=1}^K e^{\mathbf{w}_k^T \mathbf{x}_i}})\end{aligned}$$

(ii) 下面的代码用numpy库实现了softmax函数

```

1 import numpy as np
2 def softmax(x):
3     return np.exp(x)/np.sum(np.exp(x))
4
5 x = np.array([10,2,1000,1000])
6 print(softmax(x))
7 print(x-x.max())
8 print(softmax(x-x.max()))

```

输出结果为

```

1 [ 0.  0. nan nan]
2 [-990 -998  0  0]
3 [0.  0.  0.5  0.5]

```

原本计算由于每一项数值大小差异过大，导致第三四项计算指数时数值爆炸，产生nan，由于 $\sigma(\mathbf{z})_k = \frac{\exp(\mathbf{z}_k)}{\sum_{j=1}^K \exp(\mathbf{z}_j)} = \frac{\exp(\mathbf{z}_k - M)}{\sum_{j=1}^K \exp(\mathbf{z}_j - M)} = \sigma(\mathbf{z} - M\mathbf{1})_k$ 因此可以考虑将每一项数值都减去这组数列的最大值，从而得到对数值计算更友好的结果。

- (iii) 在hw4.ipynb 中我实现了相应的算法，学习率为0.01，迭代10000步后loss图如下，最终在训练集和测试集上的准确率分别为87.38% 和82.56%

